

1. Purpose of the Week 5 Check

The checks focus on schema constraints, data quality, normalization up to Third Normal Form (3NF), and whether the database structure can support large analytical queries.

2. Database Tables Checked

Table	Primary Key	Normalization Result
City	city_id	3NF
Platform	platform_id	3NF
Regulation	regulation_id	3NF
CityStatistics	(city_id, year)	3NF
RentalListing	listing_id	3NF
CityRegulation	(city_id, regulation_id)	3NF

3. Normalization Check

The database satisfies 1NF. The attributes are intended to contain atomic values and there are no repeating groups within the tables. Each record is identified by a primary key or, a composite primary key.

The database satisfies 2NF. Tables with single column primary keys have no partial dependency. For CityStatistics, the composite key is (city_id, year), and the attributes depend on the combination of city and year. For CityRegulation, the composite key is (city_id, regulation_id), and start_date and end_date depend on the relationship between the city and regulation.

The database satisfies 3NF. No transitive dependency between non key attributes. City names, platform names and regulation names are stored in their own tables and referenced through foreign keys instead of being stored in RentalListing, CityStatistics or CityRegulation.

4. Functional Dependency Check

Table	Main Functional Dependency
City	city_id → city_name
Platform	platform_id → platform_name
Regulation	regulation_id → regulation_name
CityStatistics	(city_id, year) → population, total_housing_units, social_housing_units, avg_rent, homeless_count
RentalListing	listing_id → city_id, platform_id, rental_type, price_amount, price_period, snapshot_date
CityRegulation	(city_id, regulation_id) → start_date, end_date

5. Schema and Data Quality Checks

Check	Validation
Primary keys	Check for duplicate IDs before insertion. Composite keys such as (city_id, year) must also be unique.
Foreign keys	Check that every city, platform and regulation referenced by another table exists in the corresponding parent table.
Duplicate observations	Check for duplicate city/year statistics and duplicate city/regulation relationships.
Missing values	Identify how missing values are represented in the

	source data and convert them consistently before insertion.
Dates	Convert dates to a consistent YYYY-MM-DD representation and reject invalid dates.
Naming conventions	Standardize city, platform and regulation names before mapping them to IDs.
Numeric values	Check that population, housing units, rents and other numerical attributes are stored as valid numeric values.

6. Identified Data Quality Issue

The existing mock data contains an invalid date value: 202-09-16. The snapshot_date attribute is defined as a DATE, so this value does not follow the expected YYYY-MM-DD format. The record should be corrected or rejected during data cleaning before insertion.

This demonstrates why validation against real world data is important: source data cannot be assumed to conform automatically to the database schema.

7. Real World Data Transformation Requirements

Before inserting real world data, the following transformations should be documented and applied:

- Map textual city names from the source data to the corresponding city_id values.
- Map platform names to platform_id values rather than storing platform names repeatedly in RentalListing.
- Standardize missing value representations such as blank cells, NULL like strings, or special codes.
- Convert all dates to a consistent database DATE format.
- Identify and remove or resolve duplicate records.
- Standardize inconsistent city, platform and regulation names.
- Validate numerical fields such as rent, population and housing unit counts.
- Reject records that violate primary key, foreign key, date or other schema constraints unless they can be cleaned safely.

8. Constraint Validation Queries

The following checks can be run after importing the real world data. A query returning no rows is normally the expected result for duplicate key and orphan record checks.

Duplicate CityStatistics observations

```
SELECT city_id, year, COUNT(*)
```

```
FROM CityStatistics
```

```
GROUP BY city_id, year
```

HAVING COUNT(*) > 1;

Duplicate CityRegulation relationships

SELECT city_id, regulation_id, COUNT(*)

FROM CityRegulation

GROUP BY city_id, regulation_id

HAVING COUNT(*) > 1;

Orphan CityStatistics records

SELECT DISTINCT city_id

FROM CityStatistics

WHERE city_id NOT IN (SELECT city_id FROM City);

Orphan RentalListing city references

SELECT DISTINCT city_id

FROM RentalListing

WHERE city_id NOT IN (SELECT city_id FROM City);

Orphan RentalListing platform references

SELECT DISTINCT platform_id

FROM RentalListing

WHERE platform_id NOT IN (SELECT platform_id FROM Platform);

9. Query Validation

The Week 3 SQL files were not included in this Week 5 document, so no claim is made that the individual Week 3 queries were executed or that their exact numerical results were reproduced. Instead, the database structure was checked to ensure that it supports the intended relationships and analytical operations.

A key consideration is incomplete real world coverage. For example, a city can exist in the City table without having a corresponding CityStatistics record for a particular year. An INNER JOIN will remove such cities from the result, while a LEFT JOIN can retain them. The appropriate JOIN therefore depends on the question being answered.

10. Example Analytical Checks

Social housing availability by city:

SELECT

c.city_name,

```

cs.year,
cs.population,
cs.social_housing_units,
ROUND(
    cs.social_housing_units * 100.0 / cs.population,
    2
) AS social_housing_percentage
FROM City c
JOIN CityStatistics cs
    ON c.city_id = cs.city_id
ORDER BY cs.year DESC, social_housing_percentage DESC;

```

Rental listings by city and platform:

```

SELECT
    c.city_name,
    p.platform_name,
    rl.rental_type,
    rl.price_amount,
    rl.price_period,
    rl.snapshot_date
FROM RentalListing rl
JOIN City c
    ON rl.city_id = c.city_id
JOIN Platform p
    ON rl.platform_id = p.platform_id
ORDER BY c.city_name, p.platform_name, rl.price_amount;

```

11. Results and Interpretation

- The database schema is structurally normalized to 3NF.
- The separation of City, Platform and Regulation prevents unnecessary duplication and transitive dependencies.

- The composite keys in CityStatistics and CityRegulation correctly represent observations and many-to-many relationships.
- Real world data requires validation before insertion because source values may contain invalid dates, missing values, duplicates or inconsistent names.
- The existing mock data already revealed an invalid date, confirming the need for data quality validation.
- Queries using INNER JOIN may exclude entities for which the real world dataset has no observation. LEFT JOIN should be considered when incomplete coverage must remain visible.
- The database can support meaningful analysis of social housing availability, rental prices, cities, platforms and regulations after data cleaning and normalization.

12. Limitations and Future Work

The main limitation of the current validation is that the exact Week 3 SQL queries and their result sets were not included in this report. Therefore, this document validates the schema, normalization and general query capability rather than claiming execution results for every Week 3 query.

Future work should include running every Week 3 query against the fully populated database, recording the resulting rows, checking whether the results answer the intended business questions, and adapting queries where real world data introduces missing observations or different naming and date conventions.

13. Conclusion

The database design holds up well against the expected challenges of real world data. The schema remains normalized to 3NF, primary and foreign key relationships provide clear integrity constraints, and the separation of entities avoids unnecessary duplication. The main practical work required when integrating real world data is cleaning and transformation: handling missing values, correcting date formats, resolving duplicates, standardizing names and ensuring that all foreign key references are valid.

The validation also shows that meaningful analytical queries can be performed once the source data has been transformed into the normalized schema. The database should therefore be considered suitable for the Week 5 real world data evaluation, with the exact Week 3 query results to be added separately when those SQL files are available.